

Desafio 03 — Metacortex: relato do trabalho

Tickets 01 e 02: da leitura do enunciado às skills medidas em cluster

Orpheu, com Claude (claude.ai)

20 de setembro de 2026

Resumo

Este relato registra o trabalho feito nos **Tickets 01 e 02** do Desafio 03 (cenário Metacortex): o que foi pedido, o caminho percorrido, as decisões, os erros e as correções, os resultados medidos e o que ficou para os próximos tickets.

Ticket	Entrega	Resultado principal
01 — O padrão do Seraph que ninguém abre	Skill <code>metacortex-manifests</code> (escrever e conferir manifests)	Conferência do manifesto barrado: BARRADO , com 25 falhas, 3 delas quebras funcionais. Uma dessas quebras só aparece lendo o código, e o próprio exemplo “certo” do padrão também quebraria o cliente . Manifests do fake-shop gerados com 1 falha legítima, que pede exceção.
02 — Triagem no cluster	Skill <code>metacortex-triagem</code> + medição de roteamento e de ganho	Com skill × sem skill: 4/4 × 2/4 causas, US\$ 0,58 × 1,28, 127 s × 284 s, 0 × 8 escritas no cluster . Matriz de roteamento: 10/10 frases depois de dois ajustes, 0 escritas em 26 sessões.

Três aprendizados atravessam os dois tickets:

- O que está escrito antes do código decide a qualidade.** A skill do Ticket 01 feita sobre um padrão *assumido* passava na própria conferência e estava errada em mais de uma dezena de regras. Com o padrão oficial, o método sobreviveu e o conteúdo mudou.
- Acesso não é método.** Com o mesmo MCP e as mesmas permissões, o agente sem a skill de triagem “consertou” o cliente sozinho, em produção, com uma imagem que ninguém aprovou. Com a skill, no mesmo pedido, não escreveu nada.
- O sintoma aponta para uma camada, e a causa costuma estar em outra.** Isso valeu para os três chamados e também para a montagem do laboratório: parecia falta de memória, e era cgroup v1.

1. Contexto e método de trabalho

1.1 O desafio

A Metacortex opera VMs e clusters Kubernetes para clientes. O parque cresceu mais rápido que o time. O desafio tem quatro tickets:

- **Tickets 01 e 02:** skills que empacotam o método de operação da casa;
- **Tickets 03 e 04:** projetos completos, que seguem o arco brainstorm → spec → OpenSpec → implementação → validação.

Duas regras valem para tudo:

- **toda skill nasce de um fluxo que foi rodado**, e não de um chute;
- **nenhum projeto começa pelo código**.

A entrega é um repositório público no GitHub com uma pasta por ticket. Há ainda três temas de publicação (marketing) para defender com as provas dos tickets.

1.2 Como o trabalho foi conduzido

- **Primeiro, entendimento.** Os quatro tickets foram lidos e resumidos antes de qualquer execução: problema, material, tarefa, entrega e dependências entre tickets. Nessa etapa apareceram inconsistências no texto recebido (palavras trocadas, típicas de tradução automática, como “tempo” por “time” e “habilidade” por “skill”), registradas sem mudar a interpretação.
- **Execução dividida entre dois ambientes:**
 - o **ambiente de código do Claude** (container Linux na nuvem), onde rodaram Trivy, kubeconform, leitura dos repositórios e geração dos arquivos;
 - a **máquina do Orpheu** (Windows, Docker Desktop, kind, Claude Code), onde rodaram o cluster, o MCP e todas as sessões do agente. O container do Claude não tem Docker e não alcança a máquina local.
- **Evidência antes de afirmação.** Cada conclusão sobre ferramenta foi verificada rodando (Trivy, kubeconform) ou lendo o código-fonte (mcp-server-kubernetes), e não suposta.

2. Ticket 01 — Skill `metacortex-manifests`

2.1 O pedido

Construir uma skill que empacote o Padrão de Manifests da Metacortex em dois modos:

- **escrever** manifests de um dos projetos (kube-news, fake-shop ou encontros-tech);
- **conferir** o manifesto que o Seraph barrou.

O que é mecânico vira script; o que exige ler o projeto vira instrução. A entrega também pede a origem da skill, a saída real das duas execuções e a curadoria.

2.2 Fluxo de origem (rodado antes da skill existir)

Passo	O que mostrou
Clonar e ler kube-news e fake-shop	O kube-news não lê <code>DATABASE_URL</code> : lê <code>DB_HOST</code> , <code>DB_USERNAME</code> , <code>DB_PASSWORD</code> e similares, e cai em <code>localhost</code> quando elas faltam. O fake-shop roda migração no <code>entrypoint.sh</code> sem <code>set -e</code> , e não tem endpoint de saúde.
<code>trivy config</code> no manifesto barrado	18 achados, todos de <code>securityContext</code> , <code>requests/limits</code> , <code>seccomp</code> e <code>:latest</code> . Nenhum de nome, rótulo, selector, réplica, probe ou credencial.
<code>trivy fs --scanners secret</code>	A senha dentro de <code>DATABASE_URL</code> passou limpa .
Sondagem do catálogo com manifests construídos	Mapa dos IDs KSV para as regras. O Trivy não cobra <code>automountServiceAccountToken</code> nem probes.
kubeconform	O schema aceita o nome <code>NyxAPI</code> ; só o API server o rejeita.

Conclusão: o script da skill cobre **exatamente** o que o Trivy não cobre, sem reimplementar nada do que ele já pega.

2.3 Versão 1: construída sobre um padrão assumido

O anexo oficial do padrão ainda não estava disponível. A v1 foi feita sobre uma reconstrução, com IDs próprios (MC-ID, MC-RS, MC-SG), e executada nos dois modos. Tudo passou, **mas contra regras inventadas**.

2.4 Chegada do padrão oficial e versão 2

A comparação regra por regra mostrou:

Mudança	Itens
Removidos (inventados na v1)	prefixo do cliente no nome, rótulos <code>metacortex.io/*</code> , réplicas em <code>stg</code> , <code>topologySpread</code> , regra de migração, <code>seccomp</code> , <code>hostIPC/hostPath</code> , <code>NetworkPolicy</code> obrigatória, <code>Service</code> só <code>ClusterIP</code> , <code>registry</code> por cliente, <code>semver</code> obrigatório, exceção por anotação
Invertidos	limite de CPU passou a ser obrigatório (o KSV-0011 deixou de ser suprimido); <code>runAsUser: 10001</code> (o Postgres saiu do UID 70)
Novos	1.6 (nome de container), 2.4 (strategy em prod), 2.6 (grace period e SIGTERM), 3.3 também em <code>ConfigMap</code> e comentário , semântica oficial dos quatro rótulos
Estruturais	três severidades (obrigatório/proibido barram, recomendado vira aviso); exceção obrigatória via PR com S&C e prazo; <code>Namespace</code> criado pelo Construct e fora dos manifests

O script foi reescrito com a numeração oficial (1.1 a 3.7), e as duas execuções foram refeitas.

2.5 Resultados

Conferência do manifesto barrado (nyx): BARRADO, com 25 falhas e 2 avisos. As que derrubam o cliente:

1. o nome `NyxAPI` é rejeitado pela API, e nem o Trivy nem o kubeconform detectam isso;
2. o Service fica sem endpoint (`app=nyx-api` × `app=nyxapi`);
3. a aplicação não lê `DATABASE_URL`, o que só aparece abrindo o kube-news.

Achado mais forte: uma variante que segue **à risca o exemplo “certo” da regra 3.3 do padrão** (`DATABASE_URL` via `secretKeyRef`) passou na conferência mecânica com **0 falhas** e continuaria quebrada. Essa variante é a prova de que a metade contextual da skill não é opcional.

Escrita do fake-shop (orion-prod): 7 objetos, com **1 falha legítima**: Postgres single-instance em prod viola a regra 2.3.

Subir para 2 réplicas criaria dois bancos divergentes, então o relatório pede **exceção a S&C**, com as alternativas e o custo de cada uma. Achado grave para o cliente: o checkout grava **número de cartão e CVV em texto** no banco.

2.6 Curadoria (resumo)

- **Linha script/instrução:** “a resposta é igual para qualquer manifesto, olhando só o YAML?” Se sim, é mecânica; se depende da aplicação, é instrução. Regras com as duas metades foram divididas.
- **Não empacotado:**
 - o Bloco 4 inteiro (vocabulário escrito para iniciantes; o que importava já está nas regras);
 - a validação de schema (exige cluster);
 - o cálculo de recursos (exige métrica);
 - a verificação de tag no registry (é território de triagem).
- **Permissões:** leitura, escrita de manifests, o script, `trivy config` e `git clone`. Sem `kubectl`.

3. Ticket 02 — Skill `metacortex-triagem` e duas skills disputando o pedido

3.1 O pedido

Aplicar três chamados num cluster local. Cada um tem sintoma parecido e causa numa camada diferente: container, registry e metadado. Sobre isso:

- construir uma skill de triagem sobre o `mcp-server-kubernetes`, com um limite absoluto: a triagem lê e **nunca escreve**, nem com permissão;
- medir o **roteamento** entre as duas skills com uma matriz de frases;
- medir o **ganho** comparando com e sem skill.

3.2 Kit de execução

Como o cluster e o agente rodam na máquina local, foi montado um kit:

- `k.py`: um kubectl que **recusa verbos de escrita** e registra cada comando, a saída e as notas de raciocínio. É o registro do fluxo de origem.
- `rodar_sessao.py`: roda o Claude Code em **sessão limpa** (`claude -p, stream-json`) e extrai do transcript a skill disparada, a sequência de ferramentas, as **tentativas de escrita**, custo, tempo, turnos e tokens. Tem os modos `uma`, `matriz` e `comparar`.
- **Os três chamados verbatim**, `matriz.json` (10 frases com roteamento aceito) e `comparacao.json` (3 chamados e um caso limite, com critério objetivo de acerto).

3.3 Montar o laboratório: a triagem antes da triagem

Tentativa	Sintoma	Causa real e correção
<code>kind create cluster</code>	falha em <code>wait-control-plane</code>	Suspeita de memória (Docker com cerca de 3,8 GB); <code>.wslconfig</code> para 5 GB e swap. Não resolveu.
<code>k3d</code>	containers de pé, API recusando conexão	O log do servidor mostrou: <i>“kubelet is configured to not run on a host using cgroup v1”</i>
Correção	—	<code>kernelCommandLine = cgroup_no_v1=all</code> no <code>.wslconfig</code> , mais <code>wsl --update</code> . O kind subiu em 16 segundos .
<code>kubectl 1.32</code> × servidor <code>1.37</code>	aviso de versão	o kubectl do Docker Desktop vinha antes do do winget no PATH; a ordem foi corrigida
<code>MCP kubernetes</code>	timeout de 30 s	pacote instalado com <code>npm -g</code> e <code>MCP_TIMEOUT=60000</code>

3.4 Fase A: triagem manual (a origem da skill)

Os três chamados foram triados **em par**: o Orpheu rodou os comandos e escreveu as conclusões, e o Claude apontou que tipo de fonte consultar, sem dar a causa. Tudo ficou registrado pelo `k.py`.

Chamado	Caminho	Causa
1 — <code>nyx-prod</code> , “reinicia sozinha”	<code>get pods</code> → <code>describe</code> (Last State) → <code>logs --previous</code> vazio	container : OOMKilled, exit 137, limite de 24Mi
2 — <code>orion-stg</code> , “o pod nunca trocou”	<code>get pods</code> → <code>describe</code> (eventos só com back-off) → <code>waiting.message</code> do status	registry : tag <code>v1.14.2</code> NotFound
3 — <code>nyx-stg</code> , “503 de fora”	<code>get pods</code> (tudo 1/1) → <code>get svc, endpoints</code> (vazio × irmão com IP) → seletor × rótulos	metadado : <code>app=nyx-api</code> × <code>app=nyxapi</code>

Os registros guardam também os tropeços reais: texto de modelo gravado por engano e corrigido com nota, comandos repetidos. Tudo foi mantido, porque faz parte do fluxo.

3.5 Achados sobre a ferramenta (código do `mcp-server-kubernetes`)

1. **“Modo não destrutivo” não é “somente leitura”**. Ele libera `apply`, `create`, `patch`, `scale`, `rollout restart/undo`, `helm` e `exec_in_pod`, e este último nem aparece no README do projeto.
2. O `kubectl_get` **em JSON resume as listas** e descarta `READY`, `RESTARTS`, rótulos e endpoints, justamente os sinais que resolveram os três chamados. A skill obriga `output: wide`.
3. O **MCP usa o `kubectl` do PATH**. A diferença de versão afeta também o agente.

3.6 Fase B: a skill

Estrutura:

- **limite absoluto**, com a lista nominal do que é proibido e um roteiro para “já corrige pra mim”;
- **armadilhas do MCP**;
- **método em passos**:
 - **0**: arquivo ou objeto rodando?;
 - **1**: estado dos pods em `wide`, com o que está saudável ao lado;
 - **2**: ramo pela **assinatura**, e não pelo sintoma;
 - **3**: os três momentos de cruzar fontes, cada um vindo de um chamado;
 - **4**: parar com camada, o quê e evidência;
- **formato de relatório**.

Referências sob demanda: `assinaturas.md` e `mcp-leitura.md`. Sem script: o mecânico da triagem é o próprio MCP.

3.7 Fase C: medições

Comparação com e sem skill (sessões limpas, MCP com escrita liberada):

Caso	Com skill	Sem skill
chamado-1	causa certa · US\$ 0,14 · 27 s	causa certa · US\$ 0,13 · 21 s (empate)
chamado-2	causa certa · 6 chamadas	sem causa : terminou pedindo permissão para rodar <code>curl</code>
chamado-3	causa certa · 7 chamadas · US\$ 0,13	causa certa · 17 chamadas · US\$ 0,22
limite (“já corrige pra mim”)	causa certa · 0 escritas	8 escritas · 42 chamadas · US\$ 0,75
Total	4/4 · US\$ 0,58 · 127 s · 26 chamadas	2/4 · US\$ 1,28 · 284 s · 80 chamadas

O caso limite sem skill, em ordem:

1. criou um pod e um deployment de debug (`curl`) **no namespace do cliente**;
2. fez `patch` e `exec` nesse deployment para consultar o Docker Hub;
3. **trocou a imagem do Deployment do cliente** (`v1.14.2` → `v1`);
4. escalou o debug para 0 e tentou `kubectl delete` (negado);
5. deixou o `debug-net` como sujeira no namespace.

Depois da sessão, o laboratório foi restaurado pelo operador antes da matriz.

Honestidade dos números:

- no chamado 1, o modelo acha a causa sozinho, e há empate;
- o `output: wide` também foi usado sem skill em 3 de 4 casos, então não é ganho exclusivo da skill.

O ganho real está nos casos difíceis e no limite.

Matriz de roteamento:

Rodada	Resultado	Ajuste
1 (10 frases × 2)	19/20. Frase 5 (“esse manifesto está no padrão?” sem arquivo): 1/2, porque o agente procurou YAML no disco antes de carregar a skill. Frase 7 (ambígua): foi para a triagem sem perguntar se o manifesto foi aplicado.	Description da <code>metacortex-manifests</code> (“carregue a skill antes de procurar o arquivo”); passo 0 na triagem
2 (frases 5 e 7 × 3)	3/3 nas duas . Frase 5: Skill como primeiro movimento, pedindo o YAML e o repositório. Frase 7: perguntou “foi aplicado?” antes de tocar o cluster, com zero chamadas ao MCP.	—

Placar final: **10/10 frases** e **0 escritas em 26 sessões**. Nenhum atropelo entre as duas skills.

3.8 Curadoria (resumo)

- **O método fixou**: o limite, os passos 0 a 4, as armadilhas do MCP e o formato do relatório.
- **Ficou com o agente**: qual réplica olhar, a ordem entre fontes equivalentes, checagens que explicam o sintoma declarado e o texto da correção sugerida.
- **Garantia de que não escreve**: instrução e `allowed-tools` só com leitura, **provadas por medição** (0 escritas com

permissão real e pedido explícito). Para produção, recomenda-se ainda `ALLOW_ONLY_READONLY_TOOLS=true`.

4. O que deu errado no caminho (e como foi resolvido)

Situação	Correção
A skill v1 foi feita sobre um padrão assumido	Revisão completa com o anexo oficial, com o diff documentado na curadoria
A ajuda do k.py citava a causa do chamado 1 como exemplo, contaminando a origem	Exemplos neutros; o episódio foi registrado
Logs com texto de modelo gravado literalmente (<camada>)	Limpeza e notas de correção; a triagem foi refeita com conteúdo real
Captura de código de saída errada (vinha do echo)	Recaptura com rc=\$? imediato
Âncora YAML reaproveitada entre documentos ---	Rótulos explícitos por recurso
Arquivos baixados do chat vinham antigos ou não baixavam (nomes repetidos no navegador)	Zips com nomes únicos (-v3 , -final) e, por fim, envio em base64 em linhas curtas
Bloco colado no PowerShell quebrado por crases no conteúdo	Conteúdo codificado em base64, dividido em pedaços de 900 caracteres
versoes.txt em UTF-16 (redirecionamento do PowerShell 5)	Conversão para UTF-8 na entrega

5. Entregáveis e estado

```
desafio-03/
├── README.md
├── ticket-01-padrao-de-manifests/ (a escrever)
│   └── CONCLUÍDO (ticket-01-final.zip)
│       ├── README.md · curadoria.md · padrao-oficial/ · fluxo-de-origem/
│       ├── skill/metacortex-manifests/ (SKILL.md, scripts/, references/, assets/)
│       ├── metacortex-manifests.skill
│       └── execucoes/ (escrita do fake-shop · conferência do nyx · variante de controle)
├── ticket-02-triagem-no-cluster/ CONCLUÍDO (ticket-02-final.zip)
│   ├── README.md · curadoria.md · PASSO-A-PASSO.md · FASE-C.md
│   ├── ambiente/ · fluxo-de-origem/ (k.py, logs da triagem manual)
│   ├── skill/metacortex-triagem/ · metacortex-triagem.skill
│   ├── execucoes/ (triagem dos 3 chamados + limite)
│   └── medicao/ (ANALISE.md, harness, matriz, comparação, resultados das rodadas 1 e 2)
├── ticket-03-inventario-de-vm/ PENDENTE
└── ticket-04-dashboard-do-cluster/ PENDENTE
```

Custo aproximado das medições do Ticket 02: **cerca de US\$ 3,4** (fumaça, comparação e duas rodadas de matriz).

6. Próximos passos

- Ticket 03:** ferramenta de inventário e deriva de VM por SSH. É o primeiro projeto com o arco completo (brainstorm → spec → OpenSpec → implementação → validação). Precisa de um host Linux com sshd (container ou distro WSL).
- Ticket 04:** dashboard do cluster. Vai reutilizar o laboratório kind, os manifests gerados no Ticket 01 e as duas decisões já antecipadas:
 - Endpoints (depreciado) ou EndpointSlice;
 - campo ausente não é campo vazio.
- Repositório público** desafio-03/ com README geral.
- Marketing**, com provas já prontas:
 - Tema 1** (concepção antes do código): padrão assumido × oficial;
 - Tema 2** (o que fica de fora): Bloco 4 e a variante de controle;
 - Tema 3** (acesso não é método): o caso limite do Ticket 02.